

Шаблон Use Case: разработка мини-проекта в Cursor

Аннотация

Этот документ оформлен как публичный шаблон use case для разработки мини-проекта с помощью Cursor и других LLM-инструментов. Он показывает не только итоговый результат, но и сам процесс: как зафиксировать задачу, разбить её на этапы, ограничить контекст, подготовить понятные промпты и документировать решения так, чтобы другой разработчик мог быстро воспроизвести подход.

Введение

Шаблон рассчитан на демонстрацию практики вайбкодинга в инженерном формате, а не на “магический” результат без контекста. Каждый этап оформлен одинаково:

1. Какие карточки используются.
2. Как выглядит промпт с пояснениями.
3. Какой блок можно сразу копировать в чат без комментариев.

Как использовать шаблон:

1. Замените плейсхолдеры вида [описание проекта], [список файлов], [ограничения] на данные своего кейса.
2. Для каждого этапа создавайте отдельный запрос или отдельную сессию, если контекст уже разросся.
3. После завершения этапа сохраняйте артефакт: ответ модели, принятое решение, diff, ссылку на PR или итоговый файл.
4. Если документ публикуется публично, оставляйте только полезные для читателя фрагменты: промпт, результат, краткий вывод и принятые ограничения.

Основная часть

Ниже приведён шаблон последовательной работы с LLM над мини-проектом: от формализации задачи до фиксации результата.

- Для написания промта будем использовать эти карточки:

- Ограничения: [платформа, язык, сроки, ограничения по инфраструктуре, важные запреты]
- Домен: [финтех / b2b / внутренний инструмент / cli / automation / ai workflow / ...]

Нужно:

1. Сформулировать цель проекта.
2. Подготовить краткое описание проекта.
3. Выделить 5-8 ключевых задач.
4. Зафиксировать ограничения и критерии успеха.
5. Явно перечислить допущения и открытые вопросы, если данных не хватает.
(гипотезы и допущения 10, 12)

Формат ответа: (фиксируем ожидаемую структуру ответа 04)

1. Цель
2. Краткое описание
3. Ключевые задачи
4. Ограничения
5. Критерии успеха
6. Допущения / открытые вопросы

Ограничения ответа:

- Без кода.
- Без выбора библиотек, если это ещё рано.
- Не добавляй лишнюю теорию.
- Если есть спорные моменты, помечай их как допущения, а не как факты.

3. ПРОМТ без комментариев (для копирования)

Роль: product-minded tech lead или senior engineer по домену проекта.

Цель: Превратить сырое описание задачи в понятную инженерную постановку.

Уровень детализации: кратко, но достаточно для старта разработки.

Контекст:

- Исходное описание проекта: [описание проекта от заказчика или автора]
- Ограничения: [платформа, язык, сроки, ограничения по инфраструктуре, важные запреты]
- Домен: [финтех / b2b / внутренний инструмент / cli / automation / ai workflow / ...]

Нужно:

- 1) Сформулировать цель проекта.
- 2) Подготовить краткое описание проекта.
- 3) Выделить 5-8 ключевых задач.
- 4) Зафиксировать ограничения и критерии успеха.
- 5) Явно перечислить допущения и открытые вопросы, если данных не хватает.

Формат ответа:

- 1) Цель
- 2) Краткое описание

- Без кода.
- Без выбора библиотек, если это ещё рано.
- Не добавляй лишнюю теорию.
- Если есть спорные моменты, помечай их как допущения, а не как факты.

04 - Зафиксируй формат ответа

2. Текст промта

Цель: Разбить проект на крупные инженерные этапы так, чтобы по ним можно было вести отдельные сессии в Cursor. (цель 02) Уровень детализации: обзорный план, без кода. (детализация 03)

Контекст:

- Описание проекта: [сжатое описание из этапа 1]
- Ограничения: [ключевые ограничения]
- Формат разработки: [один исполнитель / команда / pet project / публичное demo]

Нужно:

1. Разбить проект на 5-8 этапов.
2. Для каждого этапа указать цель, результат и зависимость от предыдущих шагов.
3. Отдельно отметить, какие этапы стоит делать в новой сессии.
4. В конце предложить ближайший следующий шаг. (следующий шаг 25)

Формат ответа: (фиксируем формат плана 04)

1. Название этапа
2. Цель этапа
3. Ожидаемый артефакт
4. Что должно быть готово до начала
5. Стоит ли открывать новую сессию

Ограничения ответа: (сужаем область до полезного для планирования 06)

- Без реализации.
- Без повторения полного описания проекта.
- Не дроби работу слишком мелко.
- Не предлагай этапы, не влияющие на результат demo.

3. ПРОМТ без комментариев (для копирования)

Цель: Разбить проект на крупные инженерные этапы так, чтобы по ним можно было вести отдельные сессии в Cursor.

Уровень детализации: обзорный план, без кода.

Контекст:

- Описание проекта: [сжатое описание из этапа 1]
- Ограничения: [ключевые ограничения]
- Формат разработки: [один исполнитель / команда / pet project / публичное demo]

Нужно:

- 1) Разбить проект на 5-8 этапов.
- 2) Для каждого этапа указать цель, результат и зависимость от предыдущих

шагов.

3) Отдельно отметить, какие этапы стоит делать в новой сессии.

4) В конце предложить ближайший следующий шаг.

Формат ответа:

1) Название этапа

2) Цель этапа

3) Ожидаемый артефакт

4) Что должно быть готово до начала

5) Стоит ли открывать новую сессию

Ограничения ответа:

- Без реализации.
- Без повторения полного описания проекта.
- Не дろби работу слишком мелко.
- Не предлагай этапы, не влияющие на результат demo.

3) Архитектура, структура и именование

1. Карточки

Для написания промта будем использовать эти карточки:

 Сформулируй цель

1

ПРАВИЛА

Чётко определи, зачем выполняется запрос. Укажи ожидаемый результат.

Цель должна содержать:

- что нужно получить
- зачем это нужно
- критерии успеха (если возможно)

✓ Ответы становятся целенаправленными и соответствуют ожиданиям.

УСЛОВИЯ

- Цель формулируется **явно**, до выполнения задачи
- Цель не заменяет роль

ПРИМЕРЫ ПРОМПТОВ

«Цель: оптимизировать функцию calculateTotal для обработки массивов из 10k+ элементов. Критерий успеха: время выполнения уменьшается минимум на 30%.»

«Цель: рефакторить класс UserService, выделив логику авторизации в отдельный модуль для улучшения тестируемости.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

002 / Cursor Core

 Определи уровень детализации

3

ПРАВИЛА

Задай глубину ответа явно: обзор, детали или формальные шаги.

УРОВНИ ДЕТАЛИЗАЦИИ

- **обзор** — высокоуровневое описание
- **детали** — углублённый разбор
- **формальные шаги** — пошаговая инструкция

✓ Ответ соответствует ожидаемой глубине и не перегружает деталями.

УСЛОВИЯ

- Уровень указывается **до выполнения** задачи

ПРИМЕРЫ ПРОМПТОВ

«Дай обзорное объяснение без углубления в реализацию.»

«Дай детальный разбор с примерами кода и объяснением каждого шага.»

Ты архитектор встраиваемых систем. Дай обзор архитектуры контроллера для управления шаговыми двигателями: основные блоки, интерфейсы и принципы взаимодействия. Без углубления в схемотехнику и прошивку.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

003 / Cursor Core

 Зафиксируй формат ответа

4

ПРАВИЛА

Явно укажи формат ответа: текст, список, таблица, diff, псевдокод или другой формат.

ТИПОВЫЕ ФОРМАТЫ

- **текст** — связанное описание
- **список** — нумерованный или маркер
- **таблица** — структурированные данные
- **diff** — изменения в коде
- **псевдокод** — алгоритм

✓ Ответ соответствует ожидаемому формату.

УСЛОВИЯ

- Формат указывается **до выполнения** задачи

ПРИМЕРЫ ПРОМПТОВ

«Ответь в виде нумерованного списка с краткими пояснениями.»

«Опиши алгоритм в виде псевдокода с комментариями.»

«Представь сравнение в виде таблицы, затем добавь текстовые пояснения.»

«Ответь в формате JSON с полями: status, message, data.»

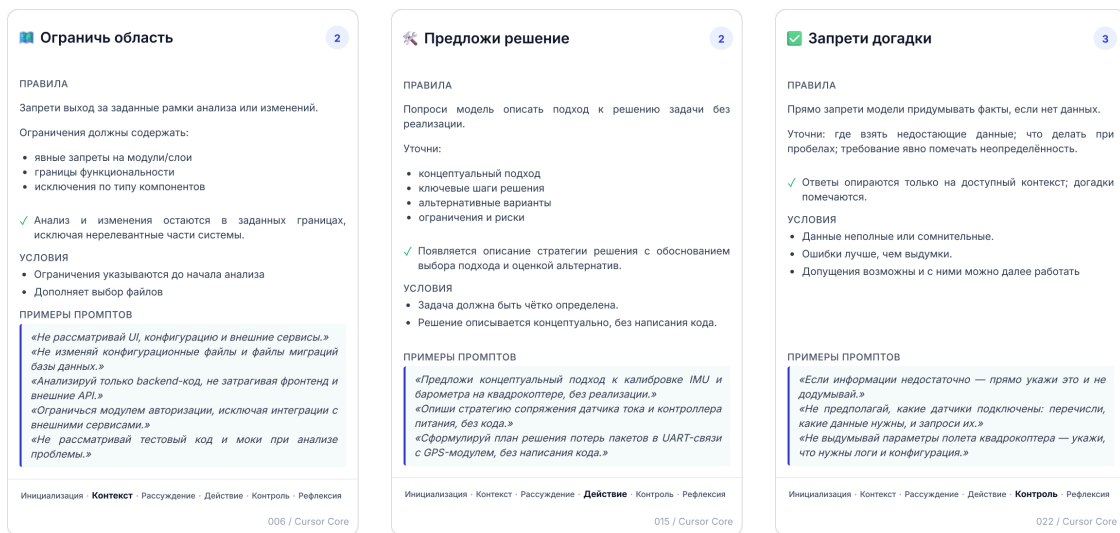
Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

004 / Cursor Core

02 - Сформулируй цель

03 - Определи уровень детализации

04 - Зафиксируй формат ответа



06 - Ограничь область

15 - Предложи решение

22 - Запрети угадывание

2. Текст промта

Цель: Определить структуру проекта, названия модулей и границы ответственности до начала кодинга. (цель 02) Уровень детализации: средний, без реализации функций. (детализация 03)

Контекст:

- Тип проекта: [cli / web / daemon / plugin / script / service / ai workflow]
- Язык / стек: [python / typescript / go / ...]
- Основные сценарии: [список 3-5 ключевых сценариев]
- Ограничения: [монорепо / минимум файлов / no framework / no db / ...]

Нужно:

1. Предложить структуру папок и файлов. (ожидаем проектное предложение от модели 15)
2. Дать названия основным модулям, классам или сервисам.
3. Кратко описать ответственность каждого файла.
4. Отдельно указать entry point, конфигурацию, тесты и документацию.
5. Если данных недостаточно, явно напиши, чего не хватает, вместо угадывания. (запрет угадывания 22)

Формат ответа: (фиксируем структуру результата 04)

1. Название проекта
2. Структура репозитория
3. Таблица: файл -> назначение
4. Список спорных мест / вопросов

Ограничения ответа: (сужаем ответ до архитектуры и именования 06)

- Без кода.
- Не перечисляй лишние файлы “на всякий случай”.
- Не описывай внутреннюю реализацию алгоритмов.

- Предлагай структуру, которую реально удобно показать публично.

3. ПРОМТ без комментариев (для копирования)

Цель: Определить структуру проекта, названия модулей и границы ответственности до начала кодинга.

Уровень детализации: средний, без реализации функций.

Контекст:

- Тип проекта: [cli / web / daemon / plugin / script / service / ai workflow]
- Язык / стек: [python / typescript / go / ...]
- Основные сценарии: [список 3-5 ключевых сценариев]
- Ограничения: [монорепо / минимум файлов / no framework / no db / ...]

Нужно:

- 1) Предложить структуру папок и файлов.
- 2) Дать названия основным модулям, классам или сервисам.
- 3) Кратко описать ответственность каждого файла.
- 4) Отдельно указать entry point, конфигурацию, тесты и документацию.
- 5) Если данных недостаточно, явно напиши, чего не хватает, вместо угадывания.

Формат ответа:

- 1) Название проекта
- 2) Структура репозитория
- 3) Таблица: файл -> назначение
- 4) Список спорных мест / вопросов

Ограничения ответа:

- Без кода.
 - Не перечисляй лишние файлы "на всякий случай".
 - Не описывай внутреннюю реализацию алгоритмов.
 - Предлагай структуру, которую реально удобно показать публично.
-

4) Исследование сложного этапа и выбор варианта

1. Карточки

Для написания промта будем использовать эти карточки:

 Проверь гипотезы 3 ПРАВИЛА Отделяй предположения от фактов при анализе проблемы. Требуй: - явное выделение гипотез - различение фактов и предположений - проверку гипотез на обоснованность ✓ Модель различает известное и предполагаемое, снижая риск необоснованных выводов. УСЛОВИЯ - Используется при неполной информации или неоднозначности - Эффективна с пошаговым рассуждением ПРИМЕРЫ ПРОМПТОВ «Если ты предполагаешь причину, явно пометь это как гипотезу.» «При анализе проблемы явно разделяй: что известно точно (факты), что предполагается (гипотезы), и почему каждая гипотеза имеет смысл.» «Если данных недостаточно для точного вывода, явно укажи это как гипотезу и объясни, почему ты её выдвигашь.» Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия 010 / Cursor Core	 Сравни альтернативы 4 ПРАВИЛА Запрашивай не одно, а несколько решений для задачи. Требуй: - предложение 2-3 альтернатив - сравнение по критериям - оценку рисков каждого варианта ✓ Пользователь получает выбор решений с пониманием компромиссов, что повышает качество принятия решений. УСЛОВИЯ - Используется для задач, где есть несколько возможных подходов - Требует явных критериев сравнения ПРИМЕРЫ ПРОМПТОВ «Предложи 2-3 возможных решения и сравни их по рискам.» «Предложи 3 варианта реализации и сравни их по производительности, сложности и поддерживаемости.» «Предложи 2-3 варианта подключения датчиков к микроконтроллеру и сравни их по надёжности, стоимости и сложности реализации.» Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия 011 / Cursor Core	 Объясняй допущения 3 ПРАВИЛА Требуй явного указания всех допущений, которые модель делает при анализе или решении задачи. Требуй: - явное перечисление допущений - обоснование каждого допущения - указание влияния на результат ✓ Модель раскрывает скрытые предположения, повышая прозрачность анализа и снижая риск ошибок. УСЛОВИЯ - Используется для сложных задач, где модель может делать неясные предположения - Дополняет проверку гипотез и пошаговое рассуждение ПРИМЕРЫ ПРОМПТОВ «Укажи все допущения, которые ты делаешь при анализе.» «Явно пометь все места в анализе, где ты делаешь допущения, и объясни, как каждое допущение влияет на выводы.» «При анализе кода укажи все допущения о его использовании, входных данных и окружении, которые ты делаешь, и обоснуй их.» Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия 012 / Cursor Core
---	---	--

10 - Проверь гипотезы

11 - Сравни альтернативы

12 - Объясняй допущения

 Оцени риски 2 ПРАВИЛА Попроси Cursor выявить возможные негативные последствия предложенного решения или изменения кода. Оценка должна включать: технические риски, эксплуатационные риски, риски безопасности, влияние на другие компоненты. ✓ Видишь потенциальные проблемы до их возникновения. УСЛОВИЯ - Используется после предложения решения или генерации кода, а не вместо них - Укажи типы рисков (безопасность, производительность, совместимость) - Комбинируй с «Проверь побочные эффекты» — сначала анализ влияния, затем оценка рисков - После «Напиши код» или «Измени существующий код» помогает оценить риски перед применением ПРИМЕРЫ ПРОМПТОВ «Оцени риски потери данных и отказов при обновлении прошивки контроллера квадрокоптера.» «Оцени риски ложных срабатываний и деградации точности при слиянии данных IMU и барометра.» Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия 024 / Cursor Core
--

 Предложи решение 2 ПРАВИЛА Попроси модель описать подход к решению задачи без реализации. Уточни: - концептуальный подход - ключевые шаги решения - альтернативные варианты - ограничения и риски ✓ Появляется описание стратегии решения с обоснованием выбора подхода и оценкой альтернатив. УСЛОВИЯ - Задача должна быть чётко определена. - Решение описывается концептуально, без написания кода. ПРИМЕРЫ ПРОМПТОВ «Предложи концептуальный подход к калибровке IMU и барометра на квадрокоптере, без реализации.» «Опиши стратегию сопряжения датчика тока и контроллера питания, без кода.» «Сформулируй план решения потерь пакетов в UART-связи с GPS-модулем, без написания кода.» Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия 015 / Cursor Core

24 - Оцени риски

15 - Предложи решение

2. Текст промта

Цель: Перед реализацией сложного этапа сравнить несколько решений и выбрать одно на основании рисков и допущений. (фокус на сравнении вариантов 11)

Контекст:

- Этап: [название сложного этапа]
- Проблема: [что именно нужно решить]
- Ограничения: [производительность / UX / совместимость / sandbox / offline / ...]
- Допустимые варианты: [если уже есть 2-3 идеи, перечислить]

Нужно:

- Предложить 2-4 реалистичных варианта решения. (варианты решения 15)
- Для каждого варианта указать плюсы, минусы, риски и скрытые допущения. (гипотезы и допущения 10, 12)
- Отдельно отметить, какой вариант лучше подходит для demo / public repo.

4. В конце дать одну рекомендацию и объяснить выбор.

Формат ответа:

1. Вариант
2. Плюсы
3. Минусы
4. Риски (оценка рисков 24)
5. Допущения
6. Рекомендация

Ограничения ответа:

- Без кода.
- Не предлагай экзотические решения без явной причины.
- Если лучший вариант зависит от непроверенного факта, пометь это отдельно.

3. ПРОМТ без комментариев (для копирования)

Цель: Перед реализацией сложного этапа сравнить несколько решений и выбрать одно на основании рисков и допущений.

Контекст:

- Этап: [название сложного этапа]
- Проблема: [что именно нужно решить]
- Ограничения: [производительность / UX / совместимость / sandbox / offline / ...]
- Допустимые варианты: [если уже есть 2-3 идеи, перечислить]

Нужно:

- 1) Предложить 2-4 реалистичных варианта решения.
- 2) Для каждого варианта указать плюсы, минусы, риски и скрытые допущения.
- 3) Отдельно отметить, какой вариант лучше подходит для demo / public hero.
- 4) В конце дать одну рекомендацию и объяснить выбор.

Формат ответа:

- 1) Вариант
- 2) Плюсы
- 3) Минусы
- 4) Риски
- 5) Допущения
- 6) Рекомендация

Ограничения ответа:

- Без кода.
 - Не предлагай экзотические решения без явной причины.
 - Если лучший вариант зависит от непроверенного факта, пометь это отдельно.
-

5) Реализация отдельного этапа

1. Карточки

Для написания промта будем использовать эти карточки:

Выбери файлы

2

ПРАВИЛА

Явно укажи, какие файлы участвуют в анализе или изменении.

Указание файлов должно содержать:

- точные пути или имена файлов
- при необходимости — исключения

✓ Фокусировка на конкретных файлах ускоряет анализ и снижает риск нерелевантных изменений.

УСЛОВИЯ

- Файлы **всегда** указываются **явно** до начала анализа
- Указание файлов **не заменяет описание окружения** (карта 8)
- Можно использовать **исключения** (например, «кроме тестовых файлов»)

ПРИМЕРЫ ПРОМПТОВ

```
«Анализируй только файлы payment_service.py и models.py.»
«Работай только с файлами в директории src/auth/»: "auth_service.py", "token_manager.py".»
«Работай только с файлами драйвера датчика: sensor_driver.c, sensor_driver.h и i2c_hal.c. Не затрагивай main.c и файлы в tests/»
```

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

005 / Cursor Core

Ограничь область

2

ПРАВИЛА

Запрети выход за заданные рамки анализа или изменений.

Ограничения должны содержать:

- явные запреты на модули/слои
- границы функциональности
- исключения по типу компонентов

✓ Анализ и изменения остаются в заданных границах, исключая нерелевантные части системы.

УСЛОВИЯ

- Ограничения указываются до начала анализа
- Дополняет выбор файлов

ПРИМЕРЫ ПРОМПТОВ

```
«Не рассматривай UI, конфигурацию и внешние сервисы.»
«Не изменяй конфигурационные файлы и файлы миграции базы данных.»
«Анализируй только backend-код, не затрагивая фронтенд и внешние API.»
«Ограничься модулем авторизации, исключая интеграции с внешними сервисами.»
«Не рассматривай тестовый код и моки при анализе проблемы.»
```

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

006 / Cursor Core

Напиши код

3

ПРАВИЛА

Попроси модель сгенерировать новый код согласно описанному решению.

Уточни:

- требования к реализации
- формат и стиль кода
- обработку ошибок
- граничные случаи

✓ Появляется готовый код, соответствующий требованиям и описанному решению.

УСЛОВИЯ

- Решение должно быть описано.
- Чётко определи требования к коду.
- Используй карты контроля.

ПРИМЕРЫ ПРОМПТОВ

```
«Напиши драйвер для IMU-сенсора по спецификации: инициализация, чтение, ошибки.»
«Напиши модуль стабилизации квадрокоптера на основе описанного PID-контроллера.»
«Напиши код отроса датчиков через I2C с буферизацией и обработкой таймаутов.»
```

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

016 / Cursor Core

05 - Выбери файлы

Запроси diff

1

ПРАВИЛА

Проси выводить только изменения в формате diff без лишнего контекста.

Уточни: какие файлы или фрагменты правятся; требуется ли подсветка удаления/добавления.

✓ Видишь только разницу, что упрощает ревью и снижает шум.

УСЛОВИЯ

- Код уже изменён или сейчас будет меняться.
- Формат diff обязателен.

ПРИМЕРЫ ПРОМПТОВ

```
«Покажи изменения строго в формате diff.»
«Дай unified diff для патча фильтра высоты в прошивке автопилота.»
«Покажи diff только по правкам в коде калибровки датчика тока.»
```

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

019 / Cursor Core

19 - Запроси diff

06 - Ограничь область

Ограничь объём изменений

1

ПРАВИЛА

Задай жёсткий лимит на размер и количество правок.

Уточни: максимум строк/блоков; затрагиваемые файлы/функции; что делать при превышении лимита.

✓ Правки остаются малыми и обозримыми. Если ожидаются небольшие правки, то код не раздувается и не генерируется лишнее.

УСЛОВИЯ

- Лимит задан до начала работы.
- При превышении — остановка и отчёт.

ПРИМЕРЫ ПРОМПТОВ

```
«Не изменяй более 20 строк кода.»
«Внеси не более двух блоков изменений в код автопилота квадрокоптера; при превышении — остановка.»
«Исправь только обработку датчика давления, максимум 15 строк; остальное не трогай.»
```

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

020 / Cursor Core

20 - Ограничь изменения

16 - Напиши код

Проверь побочные эффекты

1

ПРАВИЛА

Попроси модель оценить, что ещё затрагивается изменениями.

Уточни: зависимые модули, интеграции; данные, миграции, конфигурации; кто использует изменяемый код.

✓ Видно, где могут появиться риски и что нужно проверить дополнительно.

УСЛОВИЯ

- Правки известны или запланированы.
- Требуется перечислить зоны влияния и риски.

ПРИМЕРЫ ПРОМПТОВ

```
«Перечисли, какие модули автопилота квадрокоптера затронет эта правка.»
«Укажи, какие цепочки обработки данных датчиков изменятся из-за обновления драйвера.»
«Проверь, какие конфигурации и калибровки могут пострадать при замене IMU.»
```

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

021 / Cursor Core

21 - Проверь побочные эффекты

2. Текст промта

Цель: Реализовать один конкретный этап без расползания в соседние части проекта. (держим задачу в узких границах 06)

Контекст:

- Этап: [название этапа]
- Задача: [что нужно реализовать]
- Файлы: [список файлов, которые можно менять] (заранее выбираем область изменений 05)
- Не трогать: [список файлов / частей системы, которые нельзя менять]

- Ограничения: [без рефакторинга / без новых зависимостей / только bugfix / только diff / ...]

Нужно:

1. Внести только необходимые изменения.
2. Кратко проверить, не ломают ли они соседние части. (проверка побочных эффектов 21)
3. Вернуть только diff или очень короткое summary + diff, если это важно. (diff 19)

Формат ответа:

1. Изменённые файлы
2. Unified diff
3. Риски / что проверить вручную

Ограничения ответа:

- Не переписывай весь файл, если достаточно локального изменения.
- Не меняй архитектуру этапа без прямого запроса. (ограничение масштаба изменений 20)
- Не добавляй новые файлы без необходимости.
- Перед изменением проверь побочные эффекты на затронутые сценарии. (side effects 21)
- Если нужен код, показывай именно реализацию, а не только рассуждения. (переход к кодированию 16)

3. ПРОМТ без комментариев (для копирования)

Цель: Реализовать один конкретный этап без расползания в соседние части проекта.

Контекст:

- Этап: [название этапа]
- Задача: [что нужно реализовать]
- Файлы: [список файлов, которые можно менять]
- Не трогать: [список файлов / частей системы, которые нельзя менять]
- Ограничения: [без рефакторинга / без новых зависимостей / только bugfix / только diff / ...]

Нужно:

- 1) Внести только необходимые изменения.
- 2) Кратко проверить, не ломают ли они соседние части.
- 3) Вернуть только diff или очень короткое summary + diff, если это важно.

Формат ответа:

- 1) Изменённые файлы
- 2) Unified diff
- 3) Риски / что проверить вручную

Ограничения ответа:

- Не переписывай весь файл, если достаточно локального изменения.
- Не меняй архитектуру этапа без прямого запроса.

Для написания промта будем использовать эти карточки:

- Изменённые файлы: [список файлов]
- Новый функционал: [что недавно было добавлено]
- Ожидаемое поведение: [коротко]
- Ограничения: [не трогать интерфейс / не менять контракт / не трогать unrelated code / ...]

Нужно:

1. Провести короткий review текущего решения.
2. Найти реальные риски, баги или несоответствия.
3. Если проблемы есть, предложить минимальный фикс. (точечное изменение кода 17)
4. Перед финальным ответом сделать самопроверку, что исправление не расширило область изменений без необходимости. (самопроверка 23)

Формат ответа:

1. Найденные проблемы
2. Причина
3. Минимальное исправление
4. Что проверить после фикса

Ограничения ответа:

- Не придумывай проблемы без признаков в коде или требованиях.
- Если проблем нет, так и напиши.
- Если предлагаешь фикс, он должен быть минимальным по охвату. (ограничение масштаба изменений 20)
- После фикса отдельно проверь, не задел ли он соседние сценарии. (проверка побочных эффектов 21)

3. ПРОМТ без комментариев (для копирования)

Цель: Найти слабые места в уже предложенном решении и исправить только подтверждённые проблемы.

Контекст:

- Изменённые файлы: [список файлов]
- Новый функционал: [что недавно было добавлено]
- Ожидаемое поведение: [коротко]
- Ограничения: [не трогать интерфейс / не менять контракт / не трогать unrelated code / ...]

Нужно:

- 1) Провести короткий review текущего решения.
- 2) Найти реальные риски, баги или несоответствия.
- 3) Если проблемы есть, предложить минимальный фикс.
- 4) Перед финальным ответом сделать самопроверку, что исправление не расширило область изменений без необходимости.

Формат ответа:

- 1) Найденные проблемы
- 2) Причина

- Не придумывай проблемы без признаков в коде или требованиях.
- Если проблем нет, так и напиши.
- Если предлагаешь фикс, он должен быть минимальным по охвату.

2. Текст промта

Цель: Добавить тесты и проверить, что новый этап не сломал соседнее поведение. (фокусируемся на тестовом контуре 18)

Контекст:

- Production-файлы: [список production файлов]
- Тестовые файлы: [список тестовых файлов] (явно выбираем, какие тесты и модули участвуют 05)
- Новый функционал: [что именно появилось]
- Риски: [что особенно важно не сломать]

Нужно:

1. Предложить или написать минимально достаточный набор тестов.
2. Покрыть позитивные, негативные и граничные сценарии.
3. Отдельно указать побочные эффекты, которые стоит проверить вручную. (проверка побочных эффектов 21)
4. Если тесты не нужны или их некуда добавлять, объяснить почему. (самопроверка на уместность запроса 23)

Формат ответа:

1. Какие кейсы покрываем
2. Какие файлы тестов меняем
3. Diff или preview новых тестов
4. Что проверить руками

Ограничения ответа: (сужаем область до тестов и верификации 06)

- Не меняй production-код, если запрос только на тесты.
- Не дублируй уже существующие кейсы без необходимости.
- Фокус на реальных сценариях использования, а не на искусственном покрытии строк.

3. ПРОМТ без комментариев (для копирования)

Цель: Добавить тесты и проверить, что новый этап не сломал соседнее поведение.

Контекст:

- Production-файлы: [список production файлов]
- Тестовые файлы: [список тестовых файлов]
- Новый функционал: [что именно появилось]
- Риски: [что особенно важно не сломать]

Нужно:

- 1) Предложить или написать минимально достаточный набор тестов.
- 2) Покрыть позитивные, негативные и граничные сценарии.
- 3) Отдельно указать побочные эффекты, которые стоит проверить вручную.
- 4) Если тесты не нужны или их некуда добавлять, объяснить почему.

- 1) Какие кейсы покрываем
- 2) Какие файлы тестов меняем
- 3) Diff или preview новых тестов
- 4) Что проверить руками

- Не меняй production-код, если запрос только на тесты.
- Не дублируй уже существующие кейсы без необходимости.
- Фокус на реальных сценариях использования, а не на искусственном покрытии строк.

1. Карточки

4. Зафиксируй формат ответа ПРАВИЛА Явно укажи формат ответа: текст, список, таблица, diff, псевдокод или другой формат. ТИПОВЫЕ ФОРМАТЫ текст — связанное описание список — нумерованный или маркер таблица — структурированные данные diff — изменения в коде псевдокод — алгоритм ✓ Ответ соответствует ожидаемому формату. УСЛОВИЯ - Формат указывается до выполнения задачи	2. Проведи самопроверку ПРАВИЛА Попроси Cursor проверить свой ответ на ошибки и несоответствия. Проверка должна включать: логические противоречия, соответствие требованиям, полноту ответа, корректность примеров кода. ✓ Модель сама находит и исправляет ошибки до финального ответа. УСЛОВИЯ - Используется после основного ответа, а не вместо - Укажи конкретные критерии проверки (логика, синтаксис, соответствие требованиям) - Комбинируй с «Проверь побочные эффекты» — сначала самопроверка, затем анализ влияния - После «Напиши код» или «Измени существующий код» помогает выявить ошибки до применения	1. Предложи следующий шаг ПРАВИЛА Попроси Cursor предложить логичный следующий шаг после выполненной работы. Предложение должно включать: что делать дальше, почему это логично, какие ресурсы могут понадобиться. ✓ Получаешь план продолжения работы без лишних размышлений. УСЛОВИЯ - Используется после завершения основного действия, а не вместо него - Пользана для сложных или многоэтапных проектов - Укажи контекст (тестирование, документация, интеграция), если важны конкретные аспекты - Для кода комбинируй с «Сгенерируй тесты» — следующий шаг может быть тестирование - После «Напиши код» или «Измени существующий код» помогает определить, что делать дальше
ПРИМЕРЫ ПРОМПТОВ «Ответь в виде нумерованного списка с краткими пояснениями.» «Опиши алгоритм в виде псевдокода с комментариями.» «Представь сравнение в виде таблицы, затем добавь текстовые пояснения.» «Ответь в формате JSON с полями: status, message, data.»	ПРИМЕРЫ ПРОМПТОВ «Проверь свой ответ на логические ошибки и несоответствия.» «Проверь свой ответ на неточности в интерфейсе SPI для датчика и несоответствия требованиям тайминга.»	ПРИМЕРЫ ПРОМПТОВ «Предложи логичный следующий шаг после этого изменения.» «Предложи следующий шаг после отладки I2C-связи между контроллером и датчиком.»
Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия	Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия	Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

23 - Самопроверка

25 - Предложи следующий шаг

Суммируй результат

1

ПРАВИЛА

Попроси Cursor кратко суммировать итог выполненной работы в структурированном виде. Итог должен включать: ключевые выводы, выполненные действия, важные детали

✓ Получаешь структурированный итог без необходимости перечислять весь ответ.

УСЛОВИЯ

- Используется **после завершения основного действия**, а не вместо него
- Не заменяет детальный ответ — это краткое резюме для быстрого понимания
- Укажи формат (список, таблица, количество пунктов), если важен конкретный вид представления
- В многоэтапных проектах помогает фиксировать прогресс на каждом этапе

ПРИМЕРЫ ПРОМПТОВ

«Кратко суммируй итог анализа в 3-5 пунктах.»
«Дай итог настройки связи датчика с МК: 3 пункта, ключевые выводы.»
«Кратко подведи итоги тестов автопилота: 5 пунктов, риски и действия.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · **Рефлексия**

026 / Cursor Core

Оцени риски

2

ПРАВИЛА

Попроси Cursor выявить возможные негативные последствия предложенного решения или изменения кода.

Оценка должна включать: технические риски, эксплуатационные риски, риски безопасности, влияние на другие компоненты.

✓ Видишь потенциальные проблемы до их возникновения.

УСЛОВИЯ

- Используется **после предложения решения** или генерации кода, а не вместо них
- Укажи типы рисков (безопасность, производительность, совместимость)
- Комбинируй с «Проверь побочные эффекты» — сначала анализ влияния, затем оценка рисков
- После «Напиши код» или «Измени существующий код» помогает оценить риски перед применением

ПРИМЕРЫ ПРОМПТОВ

«Оцени риски потери данных и отказов при обновлении прошивки контроллера квадрокоптера.»
«Оцени риски ложных срабатываний и деградации точности при слиянии данных IMU и барометра.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · **Рефлексия**

024 / Cursor Core

26 - Подведи итог

24 - Оцени риски

2. Текст промта

Цель: Подготовить краткий публичный итог по этапу или по всему mini-project use case. (задача на итоговое резюме 26)

Контекст:

- Что было сделано: [список изменений]
- Какие файлы затронуты: [список файлов]
- Какие проверки выполнены: [тесты / ручные проверки / smoke]
- Что не сделано: [если есть ограничения]

Нужно:

1. Кратко зафиксировать результат.
2. Перечислить проверенные сценарии.
3. Указать оставшиеся риски и ограничения. (оценка рисков 24)
4. Предложить следующий шаг для развития проекта или документации. (следующий шаг 25)

Формат ответа: (фиксируем финальную структуру summary 04)

1. Что сделано
2. Что проверено
3. Ограничения / риски
4. Следующий шаг

Ограничения ответа:

- Без длинного пересказа всей истории.
- Пиши так, чтобы этот блок можно было вставить в README, issue, PR или демо-описание.
- Перед финальной формулировкой проверь, что summary не разросся и не потерял главное. (самопроверка 23)

3. ПРОМТ без комментариев (для копирования)

Цель: Подготовить краткий публичный итог по этапу или по всему mini-project use case.

Контекст:

- Что было сделано: [список изменений]
- Какие файлы затронуты: [список файлов]
- Какие проверки выполнены: [тесты / ручные проверки / smoke]
- Что не сделано: [если есть ограничения]

Нужно:

- 1) Кратко зафиксировать результат.
- 2) Перечислить проверенные сценарии.
- 3) Указать оставшиеся риски и ограничения.
- 4) Предложить следующий шаг для развития проекта или документации.

Формат ответа:

- 1) Что сделано
- 2) Что проверено
- 3) Ограничения / риски
- 4) Следующий шаг

Ограничения ответа:

- Без длинного пересказа всей истории.
- Пиши так, чтобы этот блок можно было вставить в README, issue, PR или demo-описание.

Заключение

Этот шаблон лучше подходит для публичной публикации, чем линейный “лог чата”, потому что показывает разработчику воспроизводимый процесс: какие карточки применялись, зачем был нужен конкретный промт, какой формат ответа ожидается и как фиксировать результат после каждого этапа. В таком виде документ можно использовать и как инструкцию, и как демонстрационный use case для команды.

Если нужно оформить реальный кейс, поверх этого шаблона обычно достаточно добавить:

1. Короткое описание проекта и ссылки на репозиторий.
 2. 1-2 реальных примера ответов модели вместо всех ответов подряд.
 3. Финальный summary с выводами, тестами и принятыми инженерными решениями.
-

Каталог карточек (3 в строке)

В каталоге ниже собраны карточки, на которые ссылается шаблон. Их можно использовать как справочник при подготовке собственных промптов и публичных кейсов.

 Используй роль ПРАВИЛА Назначь модели профессиональную роль перед выполнением задачи. ✓ Ответы становятся последовательными и соответствуют выбранной роли. УСЛОВИЯ - Роль указывается до основной задачи - Роль не заменяет формулировку цели - Роль должна быть профессиональной, а не оценочной ПРИМЕРЫ ПРОМПТОВ «Выступай в роли senior Python backend-разработчика с опытом поддержки legacy-систем.» «Выступай в роли senior embedded-разработчика с опытом работы с микроконтроллерами, низкоуровневым программированием и протоколами связи.» Инициализация Контекст Рассуждение Действие Контроль Рефлексия	 Сформулируй цель ПРАВИЛА Чётко определи, зачем выполняется запрос. Укажи ожидаемый результат. Цель должна содержать: - что нужно получить - зачем это нужно - критерии успеха (если возможно) ✓ Ответы становятся целенаправленными и соответствуют ожиданиям. УСЛОВИЯ - Цель формулируется явно, до выполнения задачи - Цель не заменяет роль ПРИМЕРЫ ПРОМПТОВ «Цель: оптимизировать функцию calculateTotal для обработки массивов из 10k+ элементов. Критерий успеха: время выполнения уменьшается минимум на 30%.» «Цель: рефакторь класс UserService, выделяя логику авторизации в отдельный модуль для улучшения тестируемости.» Инициализация Контекст Рассуждение Действие Контроль Рефлексия	 Определи уровень детализации ПРАВИЛА Задай глубину ответа явно: обзор, детали или формальные шаги. УРОВНИ ДЕТАЛИЗАЦИИ обзор — высокоуровневое описание детали — углублённый разбор формальные шаги — пошаговая инструкция ✓ Ответ соответствует ожидаемой глубине и не перегружает деталими. УСЛОВИЯ - Уровень указывается до выполнения задачи ПРИМЕРЫ ПРОМПТОВ «Дай обзорное объяснение без углубления в реализацию.» «Дай детальный разбор с примерами кода и объяснением каждого шага.» Ты архитектор встраиваемых систем. Дай обзор архитектуры контроллера для управления шаговыми двигателями: основные блоки, интерфейсы и принципы взаимодействия. Без углубления в схемотехнику и прошивку.» Инициализация Контекст Рассуждение Действие Контроль Рефлексия
--	--	--

01 - Используй роль

02 - Сформулируй цель

03 - Определи уровень детализации

 Зафиксируй формат ответа 4 ПРАВИЛА Явно укажи формат ответа: текст, список, таблица, diff, псевдокод или другой формат. ТИПОВЫЕ ФОРМАТЫ текст — связанное описание список — нумерованный или маркированный таблица — структурированные данные diff — изменения в коде псевдокод — алгоритм ✓ Ответ соответствует ожидаемому формату. УСЛОВИЯ - Формат указывается до выполнения задачи ПРИМЕРЫ ПРОМПТОВ «Ответь в виде нумерованного списка с краткими пояснениями.» «Опиши алгоритм в виде псевдокода с комментариями.» «Представь сравнение в виде таблицы, затем добавь текстовые пояснения.» «Ответь в формате JSON с полями: status, message, data.» Инициализация: Контекст · Рассуждение · Действие · Контроль · Рефлексия	 Выбери файлы 2 ПРАВИЛА Явно укажи, какие файлы участвуют в анализе или изменении. Указание файлов должно содержать: - точные пути или имена файлов - при необходимости — исключения ✓ Фокусировка на конкретных файлах ускоряет анализ и снижает риск нерелевантных изменений. УСЛОВИЯ - Файлы всегда указываются явно до начала анализа - Указание файлов не заменяет описание окружения (карта 8) - Можно использовать исключения (например, «кроме тестовых файлов») ПРИМЕРЫ ПРОМПТОВ «Анализируй только файлы payment_service.py и models.py.» «Работай только с файлами в директории src/auth': 'auth_service.py', 'token_manager.py'.» «Работай только с файлами драйвера датчика: sensor_driver.c, sensor_driver.h и i2c_hal.c. Не затрагивай main.c и файлы в tests!» Инициализация: Контекст · Рассуждение · Действие · Контроль · Рефлексия	 Ограничь область 2 ПРАВИЛА Запрети выход за заданные рамки анализа или изменений. Ограничения должны содержать: - явные запреты на модули/слои - границы функциональности - исключения по типу компонентов ✓ Анализ и изменения остаются в заданных границах, исключая нерелевантные части системы. УСЛОВИЯ - Ограничения указываются до начала анализа - Дополнить выбор файлов ПРИМЕРЫ ПРОМПТОВ «Не рассматривай UI, конфигурацию и внешние сервисы.» «Не изменяй конфигурационные файлы и файлы миграции базы данных.» «Анализируй только backend-код, не затрагивая фронтенд и внешние API.» «Ограничь модулем авторизации, исключая интеграции с внешними сервисами.» «Не рассматривай тестовый код и моки при анализе проблемы.» Инициализация: Контекст · Рассуждение · Действие · Контроль · Рефлексия
--	---	--

04 - Зафиксируй формат ответа

05 - Выбери файлы

06 - Ограничь область

Укажи точку входа

2

ПРАВИЛА

Фиксируй, с какого класса / функции начинать анализ или изменение.

Точка входа должна содержать:

• конкретный класс, метод или функцию

• путь к файлу (если нужно)

• контекст начала работы

✓ Анализ начинается с указанной точки, обеспечивая последовательность и логичность исследования кода.

УСЛОВИЯ

• Точка входа указывается до начала анализа

• Дополняет выбор файлов

ПРИМЕРЫ ПРОМПТОВ

«Начни анализ с метода process_payment().»

«Начни с функции handle_request() в файле api_handler.py.»

«Начни с метода validate() и проанализирую всю цепочку валидации.»

«Начни анализ с конструктора класса PaymentProcessor и проследи инициализацию зависимостей.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

007 / Cursor Core

Опиши окружение

1

ПРАВИЛА

Укажи версии, фреймворки, ограничения инфраструктуры и технологический стек проекта.

Описание окружения должно содержать:

• версии языков и библиотек

• используемые фреймворки

• ограничения инфраструктуры

• зависимости и конфигурации

✓ Решения и код соответствуют технологическому стеку проекта, исключая несовместимые варианты.

УСЛОВИЯ

• Окружение описывается до генерации кода или предложения решений

• Не заменяет выбор файлов

ПРИМЕРЫ ПРОМПТОВ

«Проект использует Python 3.11, FastAPI и PostgreSQL.»

«Архитектура системы управления квадрокоптером: Python 3.11, DroneKit, MAVLink. Полетный контроллер: Pixhawk (PX4). Связь: телеметрия по WiFi 5 GHz. Не используй ROS.

Ограничение: поддержка автономных миссий и ручного режима.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

008 / Cursor Core

Пошаговое рассуждение

2

ПРАВИЛА

Принуждай модель к явной логике при решении задачи.

Требуй:

• явное описание каждого шага

• логические связи между шагами

• промежуточные выводы

✓ Модель раскрывает ход мысли, что позволяет проверить логику и выявить ошибки рассуждения.

УСЛОВИЯ

• Используется для сложных задач, где важна прозрачность мышления

• Не заменяет проверку гипотез

ПРИМЕРЫ ПРОМПТОВ

«Разбей проблему пошагово, явно описывая каждый вывод.»

«Разбей код последовательно: сначала объясни структуру, затем логику каждого компонента, затем их взаимодействие.»

«Разбей ошибки пошагово: сначала воспроизведи проблему, затем найди место возникновения, затем определи причину, затем предложи решение.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

009 / Cursor Core

07 - Укажи entry_point

08 - Укажи окружение

09 - Пошаговое рассуждение

Проверь гипотезы

3

ПРАВИЛА

Отделяй предположения от фактов при анализе проблемы.

Требуй:

• явное выделение гипотез

• различие фактов и предположений

• проверку гипотез на обоснованность

✓ Модель различает известное и предполагаемое, снижая риск необоснованных выводов.

УСЛОВИЯ

• Используется при неполной информации или неоднозначности

• Эффективна с пошаговым рассуждением

ПРИМЕРЫ ПРОМПТОВ

«Если ты предполагаешь причину, явно пометь это как гипотезу.»

«При анализе проблемы явно разделяй: что известно точно (факты), что предполагается (гипотезы), и почему каждая гипотеза имеет смысл.»

«Если данных недостаточно для точного вывода, явно укажи это как гипотезу и объясни, почему ты её выдвигаешь.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

010 / Cursor Core

Сравни альтернативы

4

ПРАВИЛА

Запрашивай не одно, а несколько решений для задачи.

Требуй:

• предложение 2-3 альтернатив

• сравнение по критериям

• оценку рисков каждого варианта

✓ Пользователь получает выбор решений с пониманием компромиссов, что повышает качество принятия решений.

УСЛОВИЯ

• Используется для задач, где есть несколько возможных подходов

• Требует явных критериев сравнения

ПРИМЕРЫ ПРОМПТОВ

«Предложи 2-3 возможных решения и сравни их по рискам.»

«Предложи 3 варианта реализации и сравни их по производительности, сложности и поддерживаемости.»

«Предложи 2-3 варианта подключения датчиков к микроконтроллеру и сравни их по надёжности, стоимости и сложности реализации.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

011 / Cursor Core

Объясняй допущения

3

ПРАВИЛА

Требуй явного указания всех допущений, которые модель делает при анализе или решении задачи.

Требуй:

• явное перечисление допущений

• обоснование каждого допущения

• указание влияния на результат

✓ Модель раскрывает скрытые предположения, повышая прозрачность анализа и снижая риск ошибок.

УСЛОВИЯ

• Используется для сложных задач, где модель может делать неявные предположения

• Дополняет проверку гипотез и пошаговое рассуждение

ПРИМЕРЫ ПРОМПТОВ

«Укажи все допущения, которые ты делаешь при анализе.»

«Явно пометь все места в анализе, где ты делаешь допущения, и объясни, как каждое допущение влияет на выводы.»

«При анализе кода укажи все допущения о его использовании, входных данных и окружении, которые ты делаешь, и обоснуй их.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

012 / Cursor Core

10 - Проверь гипотезы

11 - Сравни альтернативы

12 - Объясняй допущения

Объясни код

2

ПРАВИЛА

Попроси модель объяснить существующий код без внесения изменений.

✓ Появляется текстовое объяснение поведения кода и его роли в системе.

УСЛОВИЯ

• Код уже существует и доступен модели (в выделенных файлах или вставлен фрагментом).

• В запросе нет требований менять код, добавлять новые функции или править баги.

• Желательно ограничить область: указать конкретный метод, класс или модуль.

• Для больших файлов карта комбинируется с картами контекста («Выбери файлы», «Огранич область»).

ПРИМЕРЫ ПРОМПТОВ

«Объясни, что делает этот метод и как он используется в системе.»

«Объясни, как этот модуль взаимодействует с датчиком высоты и какие входы/выходы он обрабатывает.»

«Объясни логику этого контроллера полёта для квадрокоптера и где он подключается к остальным компонентам.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

013 / Cursor Core

Найди проблему

2

ПРАВИЛА

Попроси модель найти баги, риски, узкие места и потенциальные ошибки.

Уточни:

• тип проблемы (баг, риск, производительность)

• условия проявления

• потенциальные последствия

✓ Модель выявляет и структурирует проблемы с указанием контекста и условий их возникновения.

УСЛОВИЯ

• Код должен быть доступен для анализа.

• Чётко определи область поиска.

• Не смешивай с исправлением.

ПРИМЕРЫ ПРОМПТОВ

«Найди риски в обработке данных IMU на квадрокоптере и опиши условия их появления.»

«Проверь интерфейс I2C датчика давления на потенциальные ошибки и укажи, когда они проявятся.»

«Найди узкие места производительности в цикле управления приводами и опиши последствия.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

014 / Cursor Core

Предложи решение

2

ПРАВИЛА

Попроси модель описать подход к решению задачи без реализации.

Уточни:

• концептуальный подход

• ключевые шаги решения

• альтернативные варианты

• ограничения и риски

✓ Появляется описание стратегии решения с обоснованием выбора подхода и оценкой альтернатив.

УСЛОВИЯ

• Задача должна быть чётко определена.

• Решение описывается концептуально, без написания кода.

ПРИМЕРЫ ПРОМПТОВ

«Предложи концептуальный подход к калибровке IMU и барометра на квадрокоптере, без реализации.»

«Опиши стратегию сопряжения датчика тока и контроллера питания, без кода.»

«Сформулируй план решения потерь пакетов в UART-связи с GPS-модулем, без написания кода.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · Рефлексия

015 / Cursor Core

13 - Объясни код

14 - Найди проблему

15 - Предложи решение

 **Напиши код**

3

ПРАВИЛА

Попроси модель сгенерировать новый код согласно описанному решению.

Уточни:

- требования к реализации
- формат и стиль кода
- обработку ошибок
- граничные случаи

✓ Появляется готовый код, соответствующий требованиям и описанному решению.

УСЛОВИЯ

- Решение должно быть описано.
- Чётко определи требования к коду.
- Используй карты контроля.

ПРИМЕРЫ ПРОМПТОВ

«Напиши драйвер для IMU-сенсора по спецификации: инициализация, чтение, ошибки.»
«Напиши модуль стабилизации квадрокоптера на основе описанного PID-контроллера.»
«Напиши код опроса датчиков через I2C с буферизацией и обработкой таймаутов.»

Инициализация · Контекст · Рассуждение · **Действие** · Контроль · Рефлексия

016 / Cursor Core

 **Измени существующий код**

3

ПРАВИЛА

Попроси модель внести точечные правки в существующий код.

Уточни:

- что именно изменить
- минимальный объём правок
- сохранение структуры файла
- совместимость с существующим кодом

✓ Появляются точечные изменения существующего кода с сохранением структуры и стиля.

УСЛОВИЯ

- Код должен быть доступен.
- Чётко определи область изменений.
- Используй карты контроля.

ПРИМЕРЫ ПРОМПТОВ

«Внеси минимальные правки в существующий код, не меняя структуру файла.»
«Исправь обработку таймаута I2C в драйвере датчика, меняя только этот метод.»
«Добавь ограничение на тягу моторов в контроллере квадрокоптера без правок соседних функций.»

Инициализация · Контекст · Рассуждение · **Действие** · Контроль · Рефлексия

017 / Cursor Core

 **Сгенерируй тесты**

2

ПРАВИЛА

Попроси модель создать тестовое покрытие для кода.

Уточни:

- тип тестов (unit, integration)
- покрываемые сценарии
- граничные случаи
- формат и фреймворк

✓ Появляется набор тестов, покрывающий указанные сценарии и граничные случаи.

УСЛОВИЯ

- Код должен быть доступен.
- Чётко определи требования к тестам.
- Укажи используемый фреймворк.

ПРИМЕРЫ ПРОМПТОВ

«Сгенерируй unit-тесты, покрывающие основной и граничные сценарии.»
«Сгенерируй unit-тесты для драйвера I2C-датчика с моками: таймаут, битый кадр, повторная инициализация.»
«Сгенерируй тесты для CAN-шины: перегрузка, конфликт идентификаторов, задержка доставки.»

Инициализация · Контекст · Рассуждение · **Действие** · Контроль · Рефлексия

018 / Cursor Core

16 - Напиши код

 **Запроси diff**

1

ПРАВИЛА

Проси выводить только изменения в формате diff без лишнего контекста.

Уточни: какие файлы или фрагменты правятся; требуется ли подсветка удаления/добавления.

✓ Видишь только разницу, что упрощает ревью и снижает шум.

УСЛОВИЯ

- Код уже изменён или сейчас будет меняться.
- Формат diff обязателен.

ПРИМЕРЫ ПРОМПТОВ

«Покажи изменения строго в формате diff.»
«Дай unified diff для патча фильтра высоты в прошивке автопилота.»
«Покажи diff только по правкам в коде калибровки датчика тока.»

Инициализация · Контекст · Рассуждение · Действие · **Контроль** · Рефлексия

019 / Cursor Core

17 - Измени код

 **Ограничь объём изменений**

1

ПРАВИЛА

Задай жёсткий лимит на размер и количество правок.

Уточни: максимум строк/блоков; затрагиваемые файлы/функции; что делать при превышении лимита.

✓ Правки остаются малыми и обозримыми. Если ожидаются небольшие правки, то код не раздувается и не генерируется лишний.

УСЛОВИЯ

- Лимит задан до начала работы.
- При превышении — остановка и отчёт.

ПРИМЕРЫ ПРОМПТОВ

«Не изменяй более 20 строк кода.»
«Внеси не более двух блоков изменений в код автопилота квадрокоптера; при превышении — остановка.»
«Исправь только обработку датчика давления, максимум 15 строк; остальное не трогай.»

Инициализация · Контекст · Рассуждение · Действие · **Контроль** · Рефлексия

020 / Cursor Core

18 - Сгенерируй тесты

 **Проверь побочные эффекты**

1

ПРАВИЛА

Попроси модель оценить, что ещё затрагивается изменениями.

Уточни: зависимые модули, интеграции; данные, миграции, конфигурации; кто использует изменяемый код.

✓ Видно, где могут появиться риски и что нужно проверить дополнительно.

УСЛОВИЯ

- Правки известны или запланированы.
- Требуется перечислить зоны влияния и риски.

ПРИМЕРЫ ПРОМПТОВ

«Перечисли, какие модули автопилота квадрокоптера затронет эта правка.»
«Укажи, какие цепочки обработки данных датчиков изменятся из-за обновления драйвера.»
«Проверь, какие конфигурации и калибровки могут пострадать при замене IMU.»

Инициализация · Контекст · Рассуждение · Действие · **Контроль** · Рефлексия

021 / Cursor Core

19 - Запроси diff

 **Запрети догадки**

3

ПРАВИЛА

Прямо запрети модели придумывать факты, если нет данных.

Уточни: где взять недостающие данные; что делать при пробелах; требование явно пометать неопределённость.

✓ Ответы опираются только на доступный контекст; догадки помечаются.

УСЛОВИЯ

- Данные неполные или сомнительные.
- Ошибки лучше, чем выдумки.
- Допущения возможны и с ними можно далее работать

ПРИМЕРЫ ПРОМПТОВ

«Если информации недостаточно — прямо укажи это и не додумывай.»
«Не предполагай, какие датчики подключены: перечисли, какие данные нужны, и запроси их.»
«Не выдумывай параметры полета квадрокоптера — укажи, что нужны логи и конфигурация.»

Инициализация · Контекст · Рассуждение · Действие · **Контроль** · Рефлексия

022 / Cursor Core

20 - Ограничь изменения

 **Проведи самопроверку**

2

ПРАВИЛА

Попроси Cursor проверить свой ответ на ошибки и несоответствия.

Проверка должна включать: логические противоречия, соответствие требованиям, полноту ответа, корректность примеров кода.

✓ Модель сама находит и исправляет ошибки до финального ответа.

УСЛОВИЯ

- Используется **после основного ответа**, а не вместо
- Укажи конкретные критерии проверки (логика, синтаксис, соответствие требованиям)
- Комбинируй с «Проверь побочные эффекты» — сначала самопроверка, затем анализ влияния
- После «Напиши код» или «Измени существующий код» помогает выявить ошибки до применения

ПРИМЕРЫ ПРОМПТОВ

«Проверь свой ответ на логические ошибки и несоответствия.»
«Проверь свой ответ на неточности в интерфейсе SPI для датчика и несоответствия требованиям тайминга.»

Инициализация · Контекст · Рассуждение · Действие · **Контроль** · Рефлексия

023 / Cursor Core

21 - Проверь побочные эффекты

 **Оцени риски**

2

ПРАВИЛА

Попроси Cursor выявить возможные негативные последствия предложенного решения или изменения кода.

Оценка должна включать: технические риски, эксплуатационные риски, риски безопасности, влияние на другие компоненты.

✓ Видишь потенциальные проблемы до их возникновения.

УСЛОВИЯ

- Используется **после предложения решения** или генерации кода, а не вместо них
- Укажи типы рисков (безопасность, производительность, совместимость)
- Комбинируй с «Проверь побочные эффекты» — сначала анализ влияния, затем оценка рисков
- После «Напиши код» или «Измени существующий код» помогает оценить риски перед применением

ПРИМЕРЫ ПРОМПТОВ

«Оцени риски потери данных и отказов при обновлении прошивки контроллера квадрокоптера.»
«Оцени риски ложных срабатываний и деградации точности при слиянии данных IMU и барометра.»

Инициализация · Контекст · Рассуждение · Действие · **Контроль** · Рефлексия

024 / Cursor Core

22 - Запрети угадывание

23 - Самопроверка

24 - Оцени риски

Предложи следующий шаг

1

ПРАВИЛА

Попроси Cursor предложить логичный следующий шаг после выполненной работы. Предложение должно включать: что делать дальше, почему это логично, какие ресурсы могут понадобиться.

✓ Получаешь план продолжения работы без лишних размышлений.

УСЛОВИЯ

- Используется **после завершения основного действия**, а не вместо него
- Полезна для сложных или многоэтапных проектов
- Укажи контекст (тестирование, документация, интеграция), если важны конкретные аспекты
- Для кода комбинируй с «Сгенерируй тесты» — следующий шаг может быть тестирование
- После «Напиши код» или «Измени существующий код» помогает определить, что делать дальше

ПРИМЕРЫ ПРОМПТОВ

«Предложи логичный следующий шаг после этого изменения.»

«Предложи следующий шаг после отладки I2C-связи между контроллером и датчиком.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · **Рефлексия**

025 / Cursor Core

25 - Предложи следующий шаг

Суммируй результат

1

ПРАВИЛА

Попроси Cursor кратко суммировать итог выполненной работы в структурированном виде. Итог должен включать: ключевые выводы, выполненные действия, важные детали

✓ Получаешь структурированный итог без необходимости перечислять весь ответ.

УСЛОВИЯ

- Используется **после завершения основного действия**, а не вместо него
- Не заменяет детальный ответ — это краткое резюме для быстрого понимания
- Укажи формат (список, таблица, количество пунктов), если важен конкретный вид представления
- В многоэтапных проектах помогает фиксировать прогресс на каждом этапе

ПРИМЕРЫ ПРОМПТОВ

«Кратко суммируй итог анализа в 3-5 пунктах.»

«Дай итог настройки связи датчика с МК: 3 пункта, ключевые выводы.»

«Кратко подведи итоги тестов автопилота: 5 пунктов, риски и действия.»

Инициализация · Контекст · Рассуждение · Действие · Контроль · **Рефлексия**

026 / Cursor Core

26 - Подведи итог